Document number: P0594R0
Audience: WG21

Ville Voutilainen
2017-02-05

Relative comparisons and std::less<T*>

Abstract

It was recently brought to my attention that depending on optimization settings, libstdc++'s std::less for pointers is not a total order. See https://gcc.gnu.org/bugzilla/show_bug.cgi?id=78420 for the related bug report.

Quite many of us have labored under the illusion that for every platform we have, a relative comparison for pointers will just work; that doesn't seem to be the case. There have been various library issues on this matter, but I think their priorities have been deemed low because, again, we thought the problem isn't 'real'.

What to do?

There have been various suggestions:

- Change the libstdc++ implementation so that it converts the pointers to uintptr_t before doing the comparison. That's a reinterpret_cast, so it makes the call operator of std::less<T*> not eligible for constant expressions.
- Change the specification of std::less<T*> so that its call operator is not declared constexpr.
- Use compiler magic in the implementation of std::less<T*> so that it can do what it does today without losing any of its capabilities.

The first option means that the comparison is never a constant expression. That's unfortunate, because for cases where the pointers point to the same constexpr array, the comparison is valid today and can produce a translation-time result.

The second option has the same issue as the first.

The third option probably works, but makes it harder to support this library with multiple compilers. The compilers would presumably need to implement the same magic.

It seems that the first and second options take us ever farther away from a situation where std::less<T*> does the same as the built-in operator does. We seemed to be going towards a direction where it does all that and more, but those suggestions seem to make the functionality of std::less<T*> a more disjoint set from the functionality of the built-in operator.

What else we could do?

There would still be the possibility of making pointer comparisons have a total order in the core language. The arguments against that seem to consist of the possibility that segmented memory models might come back, and some not-so-clear statements that some existing or forthcoming implementations use the undefined behavior to their benefit.